

Python Project - Phase 2

Design & Incubation Report

Project: Rental Management System

Prepared for submission: Tuesday, August 18, 2026

Student Name: NAHABWE DERRICK ERIC

Class / Course: CERTIFICATE IN SOFTWARE ENGINEERING WITH ARTIFICIAL INTELLIGENCE

1. Purpose

Phase 2 focuses on refining the Rental Management System before full development. The system is a Python Tkinter desktop application for managing rental properties, tenants, rent payments, dashboard summaries, and system settings. The design separates the user interface, data access, validation, and business logic so the application can be expanded safely.

Current implementation already includes a login screen, the main application window, sidebar navigation, reusable layout components, dashboard cards, reusable tables, and modal forms for adding property, tenant, and payment records in memory. The finalized Phase 2 design adds persistent MySQL storage and complete CRUD operations.

2. Refined System Architecture

The system will follow a modular architecture. Tkinter pages handle screens and user actions. Service modules contain business rules. Data modules connect to MySQL. Validation modules check data before records are saved.

Module / File	Responsibility	Interaction
main.py	Starts the application, creates the main window, sidebar, header, and page container.	Imports page classes and displays selected pages through show_page().
config/settings.py	Stores app name, window size, colors, and fonts.	Used by pages and components for consistent styling.
components/header.py	Reusable top header that displays the active page title.	Updated by main.py when navigation changes.

components/sidebar.py	Navigation menu for Dashboard, Properties, Tenants, Payments, and Settings.	Calls main.py show_page() when a user selects a page.
components/stat_card.py	Reusable dashboard summary card.	Used by dashboard_page.py to display totals.
components/simple_table.py	Reusable Treeview table for record lists.	Used by pages that display properties, tenants, and payments.
pages/base_page.py	Shared page layout, scrollable content area, and reusable buttons.	Inherited by all page modules.
pages/login_page.py	Authenticates the user before the main application layout is shown.	Calls create_app_layout() after a successful username and password check.
pages/dashboard_page.py	Displays portfolio health, occupancy, payment totals, activity, and quick actions.	Reads summary data from services after storage is connected.
pages/properties_page.py	Manages rental property records and property forms.	Calls validation and property service functions.
pages/tenants_page.py	Manages tenant records and assigned property/unit information.	Calls validation and tenant service functions.
pages/payments_page.py	Records rent payments, displays payment history, and tracks balances.	Calls validation and payment service functions.
pages/settings_page.py	Stores application settings and user preferences.	Calls settings service where required.
data/database.py	Opens MySQL connections, initializes the database, loads property options, and saves property/tenant records.	Used by pages and future service modules.

data/schema.sql	Defines the MySQL database, users, properties, tenants, and payments tables.	Imported into MySQL/XAMPP before using persistent storage.
requirements.txt	Lists required external packages for database connection.	Installs mysql-connector-python.
services/property_service.py	Proposed module: add, update, delete, search, and list properties.	Uses database.py and validators.py.
services/tenant_service.py	Proposed module: add, update, delete, search, and list tenants.	Uses database.py and validators.py.
services/payment_service.py	Proposed module: record payments, search payments, calculate balances.	Uses database.py and validators.py.
utils/validators.py	Proposed module: validates names, phone numbers, money values, dates, and IDs.	Called before data is saved or updated.

Module Interaction Flow

```

User Action in Tkinter Page      |      v Page method reads form data and calls
validation                       |      v Service module applies business rules      |
v data/database.py runs MySQL query |      v Result returns to service,
then page                        |      v Tkinter page refreshes table, dashboard, or messagebox

```

3. Detailed Flowcharts

3.1. Add Record Flowchart

```

Start      v User clicks Add button      v Display input form/modal      v User enters
record details      v Validate required fields      v If invalid: show error and return to
form      v Validate data types and ranges      v If invalid: highlight field and show
correction message      v Check duplicate ID/contact      v If duplicate: show duplicate
record error      v Save record to MySQL      v Refresh table/dashboard      v Show success
message      v End

```

3.2. Update Record Flowchart

Start v User selects existing row v If no row selected: show selection message
 v Load record into edit form v User edits details v Validate changed values v
 If invalid: keep form open and show field error v Confirm update action v If
 cancelled: close form without saving v Update record in MySQL v If record
 missing: show not found message v Refresh list and summaries v End

3.3. Delete Record Flowchart

Start v User selects a record v If no selection: show selection error v Check
 related records v If dependency exists: block delete or ask to reassign v Show
 confirmation dialog v If cancelled: return to list v Delete record from MySQL
 v If database error: show error and keep data unchanged v Refresh table/dashboard
 v End

3.4. Search / View Records Flowchart

Start v User opens target page v Load all records from database v If database
 file is missing: create database and show empty table v User enters search keyword
 v Validate keyword/filter v If keyword is empty: display all records v Query
 matching records v If no match: show record not found message v Display results
 in table v End

3.5. Exit Flowchart

Start v User clicks Exit or closes application window v Check unsaved form
 changes v If unsaved changes exist: ask user to confirm exit v Commit and close
 database connection v Destroy Tkinter window v End

4. Function-Level Design

Function Name	Purpose	Parameters	Returns / Output
RentalManagementApp.__init__()	Create the main Tkinter app window and layout.	self	Initialized application window.
create_pages()	Create page objects once and store them in a dictionary.	self	None.
show_page()	Display the selected page and update header/sidebar state.	page_name: str	None. Ignores invalid page name.
LoginPage.check_login()	Check username and password before opening the main dashboard.	self	Calls login success function or shows login error.

LoginPage.toggle_password_visibility()	Show or hide password characters on the login form.	self	None. Updates password entry display.
BasePage.create_primary_button()	Create a consistent primary action button.	parent, text, command	tk.Button instance.
BasePage.create_secondary_button()	Create a consistent secondary action button.	parent, text, command	tk.Button instance.
PropertiesPage.add_property()	Open modal form used to capture property details.	self	None. Shows modal.
PropertiesPage.save_property()	Validate property fields and save them.	entries: dict, modal	Success through table refresh or warning message.
PropertiesPage.center_modal()	Position a modal window at the center of the main app.	modal	None.
SimpleTable.set_rows()	Replace current table rows with a provided record list.	rows: list[tuple]	None. Updates Treeview.
DashboardPage.refresh_page()	Reload dashboard data and update stat cards.	self	None. Updates cards.
initialize_database()	Create the MySQL database and tables from data/schema.sql when MySQL is available.	None	True if initialized, False if unavailable.
get_property_options()	Load active properties for the Add Tenant dropdown.	None	List of property records.
add_property()	Save a new property record.	property_data: dict	New property ID or error message.
get_properties()	Load properties from MySQL for the	None	List of property records.

	Properties page.		
update_property()	Edit an existing property record.	property_id: int, updates: dict	True/False with message.
delete_property()	Remove a property if it has no blocked dependencies.	property_id: int	True/False with message.
search_properties()	Find properties by name, location, status, or ID.	keyword: str	List of matching records.
add_tenant()	Save a new tenant and link to a property/unit.	tenant_data: dict	New tenant ID or error message.
get_tenants()	Load tenant records with their linked property names.	None	List of tenant records.
record_payment()	Save rent payment and update balance history.	payment_data: dict	Payment ID or error message.
calculate_balance()	Calculate outstanding balance for a tenant.	tenant_id: int	Balance amount.
validate_required()	Check that required fields are not empty.	fields: dict	True or validation message.
validate_money()	Confirm currency values are numeric and not negative.	amount: str	Float value or error message.
validate_date()	Confirm date follows YYYY-MM-DD format.	date_text: str	date object or error message.

5. Final Data Structure / Database Schema

Final storage method: MySQL. MySQL is suitable because the project now needs relational storage that can link properties, tenants, and payments, while also supporting multi-table queries, constraints, and future expansion beyond in-memory sample data.

5.1. properties Table

Field Name	Data Type	Constraint	Description
property_id	INT	PRIMARY KEY AUTO_INCREMENT	Unique property record ID.
property_name	VARCHAR(120)	NOT NULL	Name of property.
location	VARCHAR(120)	NOT NULL	Physical location.
total_units	INT	NOT NULL DEFAULT 0	Number of rentable units.
monthly_rent	DECIMAL(12,2)	NOT NULL DEFAULT 0.00	Default monthly rent per unit.
status	VARCHAR(30)	NOT NULL DEFAULT 'Active'	Active or Inactive.
created_at	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP	Date created.

5.2. tenants Table

Field Name	Data Type	Constraint	Description
tenant_id	INT	PRIMARY KEY AUTO_INCREMENT	Unique tenant ID.
property_id	INT	FOREIGN KEY REFERENCES properties(property_id)	Selected property from the dropdown.
tenant_name	VARCHAR(120)	NOT NULL	Tenant full name.
phone	VARCHAR(40)	UNIQUE NOT NULL	Tenant phone number.
email	VARCHAR(120)	NULL	Optional tenant email.
unit_number	VARCHAR(40)	NOT NULL	Assigned room/unit.
lease_start	DATE	NULL	Lease start date.
monthly_rent	DECIMAL(12,2)	NOT NULL DEFAULT 0.00	Tenant monthly rent.
emergency_contact	VARCHAR(120)	NULL	Emergency contact details.

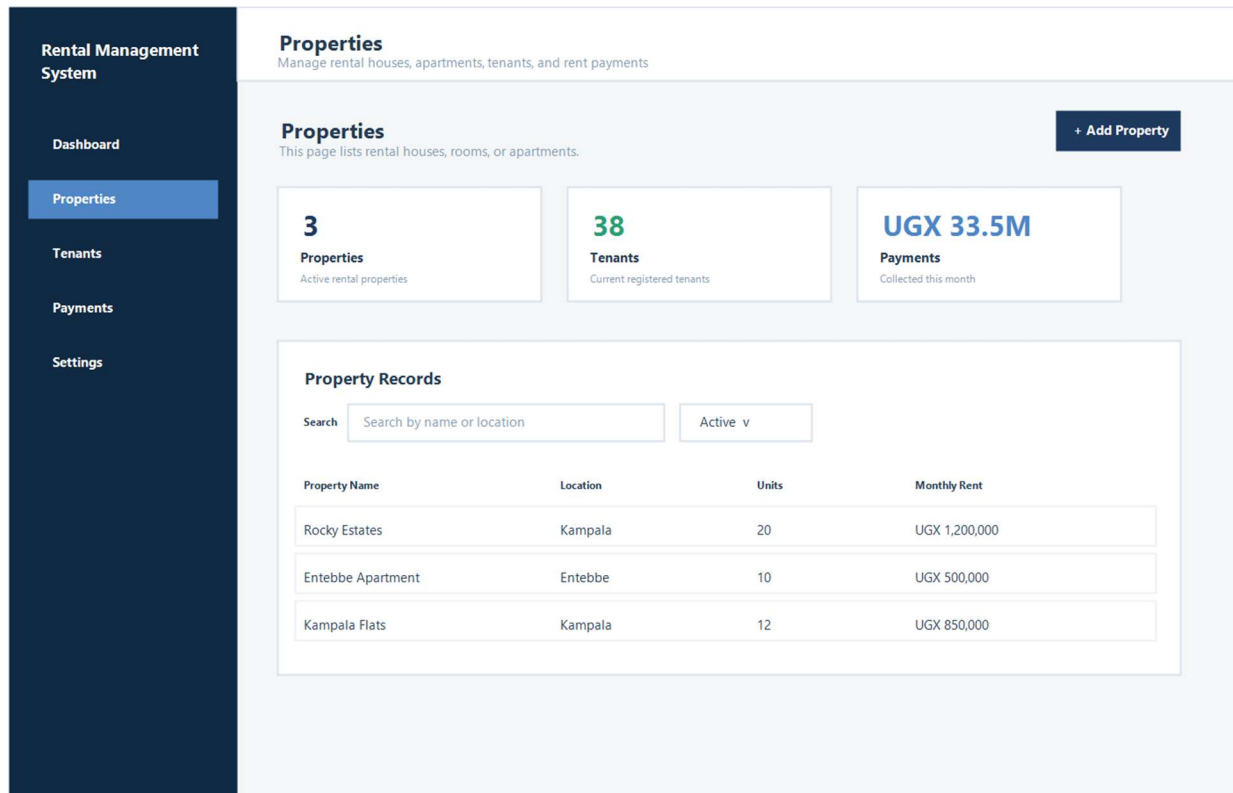
status	VARCHAR(30)	NOT NULL DEFAULT 'Active'	Active, Pending Move-in, or Notice Given.
--------	-------------	---------------------------	---

5.3. payments Table

Field Name	Data Type	Constraint	Description
payment_id	INT	PRIMARY KEY AUTO_INCREMENT	Unique payment ID.
tenant_id	INT	FOREIGN KEY REFERENCES tenants(tenant_id)	Tenant who paid rent.
amount	DECIMAL(12,2)	NOT NULL	Payment amount.
payment_date	DATE	NOT NULL	Date payment was received.
payment_method	VARCHAR(40)	NOT NULL	Cash, Mobile Money, Bank Transfer, or Card.
reference_no	VARCHAR(80)	UNIQUE	Transaction or receipt reference.
notes	TEXT	NULL	Optional payment notes.
status	VARCHAR(30)	NOT NULL DEFAULT 'Paid'	Paid, Pending, or Overdue.

5.4. Sample Record Structure

The screenshot below shows sample property records as they appear in the project interface after the database fields are connected to the table view.



6. UI Mockups and Navigation Layout

The project uses a Tkinter graphical interface. The user navigates with a left sidebar, while the top header displays the active page. The screenshots below show the properties screen and the Add Tenant form with the new property dropdown.

Main Project Screen

Rental Management System

Dashboard

Properties

Tenants

Payments

Settings

Properties

Manage rental houses, apartments, tenants, and rent payments

3

Properties

Active rental properties

38

Tenants

Current registered tenants

UGX 33.5M

Payments

Collected this month

+ Add Property

Property Records

Search

Search by name or location

Active v

Property Name	Location	Units	Monthly Rent
Rocky Estates	Kampala	20	UGX 1,200,000
Entebbe Apartment	Entebbe	10	UGX 500,000
Kampala Flats	Kampala	12	UGX 850,000

Add Tenant Form With Property Dropdown

Rental Management System

Dashboard

Properties

Tenants

Payments

Settings

Tenants

Store tenant contacts, selected property, unit, and rent status

+ Add Tenant

Tenants

This page stores tenant names, contacts, and rental details.

Add Tenant

Capture the tenant profile, selected property, assigned unit, and rent status.

Tenant Name

Sarah N.

Phone Number

+256 700 123 456

Email Address

sarah@example.com

Property

Rocky Estates - Kampala v

Unit Number

B12

Lease Start Date

2026-08-18

Monthly Rent

1200000

Status

Active v

Cancel

Save Tenant

Navigation Flow

Open App -> Login Login Success -> Dashboard Dashboard -> Properties -> Add / Update / Delete / Search Property Dashboard -> Tenants -> Add / Update / Delete / Search Tenant Dashboard -> Payments -> Record / Update / Delete / Search Payment Any Page -> Settings Any Page -> Close Window -> Confirm Exit -> End

7. Error Handling Plan

Possible Error	Handling Method	User Feedback
Empty required field	Check field before save using <code>validate_required()</code> .	Show warning: Please fill in all required details.
Incorrect data type	Convert values inside try-except blocks.	Show warning: Units must be a whole number or amount must be numeric.
Negative amount or unit count	Use range checks and database CHECK constraints.	Show warning: Value cannot be negative.
Duplicate phone or payment reference	Catch MySQL integrity errors.	Show warning: Record already exists.
Missing connector or MySQL server	Return fallback sample data and keep the app open.	Continue with sample properties until setup is complete.
Record not found	Check query result before update/delete.	Show message: Record not found.
Foreign key conflict	Use MySQL foreign key constraints and catch database errors.	Show message explaining related tenant/payment records exist.
Unexpected database failure	Use try-except-finally around database operations.	Show safe error message and keep app open.
Invalid date	Parse date using <code>datetime.strptime()</code> .	Show warning: Use YYYY-MM-DD date format.
User cancels operation	Close modal without calling <code>save/update/delete</code> .	No data is changed.

Validation Rules

- Property name, location, tenant name, phone number, amount, and payment date must not be empty.
- Units must be a whole number greater than or equal to zero.
- Money values must be numeric and greater than or equal to zero, except payments which must be greater than zero.
- Phone numbers must be unique and should follow the local/international phone format accepted by the project.
- Dates must use ISO format: YYYY-MM-DD.
- Delete actions must ask for confirmation before changing data.

8. Detailed Testing Plan

Function	Test Input	Expected Output	Type
add_property	PROP-004, Ntinda Villas, 8, 7000000	Record saved and table refreshes.	Normal
add_property	Empty property name	Error message for missing details.	Invalid
add_property	Units = abc	Error message: Units must be a whole number.	Invalid
add_property	Duplicate property code PROP-001	Duplicate record error.	Edge
update_property	Change Rocky Estates units from 20 to 22	Record updated and dashboard refreshes.	Normal
update_property	No selected row	Message asks user to select a record.	Invalid
delete_property	Delete property with no tenants	Property removed from database and table.	Normal
delete_property	Delete property with active tenants	Delete blocked or user asked to reassign tenants.	Edge
search_properties	Keyword Kampala	Only matching property records display.	Normal
search_properties	Unknown keyword ZZZ	No records found message or empty table.	Edge

add_tenant	Valid tenant linked to property 1	Tenant saved successfully.	Normal
add_tenant	Duplicate phone number	Duplicate phone warning.	Invalid
record_payment	Tenant 1, UGX 1200000, 2026-08-10	Payment saved and balance recalculated.	Normal
record_payment	Amount = -50000	Error message: payment amount must be greater than zero.	Invalid
record_payment	Date = 10/08/2026	Error message: Use YYYY-MM-DD.	Invalid
calculate_balance	Tenant with partial payment	Correct unpaid balance displayed.	Edge
check_login	Username eric and password 12345	Main dashboard opens.	Normal
check_login	Wrong password	Login Failed error message appears.	Invalid
show_page	Dashboard	Dashboard page appears and sidebar highlights Dashboard.	Normal
show_page	UnknownPage	No crash; page remains unchanged.	Edge
exit_app	Close window with no unsaved data	Database closes and app exits.	Normal
exit_app	Close window with unsaved form data	Confirmation message appears.	Edge

Testing Methods

- Manual GUI testing will confirm navigation, button actions, modal behavior, and table refreshes.
- Unit tests will check validators and service functions without opening the Tkinter interface.
- Database tests will use a test MySQL database or sample fallback data to avoid changing real records.
- Regression tests will verify that dashboard totals still update after CRUD changes.

9. Development Timeline

Stage	Main Work	Deliverables
Week 1 - Menu and Structure	Confirm navigation, page layout, reusable buttons, tables, and dashboard shell.	Stable GUI skeleton with Dashboard, Properties, Tenants, Payments, and Settings pages.
Week 2 - Data Storage	Create data/database.py, data/schema.sql, seed/sample data, and MySQL connection helpers.	rental_management_system database created with properties, tenants, and payments tables.
Week 3 - CRUD Operations	Implement add, update, delete, search, and list operations for properties and tenants.	Working property and tenant management screens.
Week 4 - Payments and Balances	Implement rent payment recording, payment history, balance calculation, and dashboard summaries.	Working payment workflow and accurate dashboard totals.
Week 5 - Validation and Error Handling	Add validators, try-except handling, confirmation dialogs, and user-friendly messages.	Reduced crashes and clear feedback for invalid actions.
Week 6 - Testing and Debugging	Run manual GUI tests, unit tests, database tests, and fix issues.	Final tested project ready for Phase 3 development/submission.

10. Conclusion

This Phase 2 design prepares the Rental Management System for organized development. The application will use Tkinter for the interface, MySQL for storage, separate service modules for business logic, and validators for clean input handling. The flowcharts, function designs, database schema, UI mockups, error handling plan, test cases, and timeline provide a clear roadmap for the next phase.